

Build Governance Platform Follow Up Review Findings

Version 1.0 | 18 September 2026 | Remediation verification review

Prepared for Freston Kenny Adedeme and the development team. Prepared by ChatGPT from the updated repository archive. This is a source review, not an independent human sign-off.

Review conclusion

The update makes useful corrections across all five original findings. However, four high-priority findings remain partially resolved. The setup documentation is substantially improved, with clean-environment verification still outstanding. None of the findings is formally closed by this report.

This follow-up supplements BGP Development Review Findings v1.0 dated 17 September 2026. It records the newer snapshot and the remaining work without overwriting the original review evidence.

Finding	Priority	Follow up status
BGP-F01 Session MFA and factor replacement	High	Partially resolved
BGP-F02 Independent compensating review	High	Partially resolved
BGP-F03 Decision concurrency protection	High	Partially resolved
BGP-F04 Lossless export and import	High	Partially resolved
BGP-F05 Setup and status guidance	Medium	Substantially addressed; verification pending

Evidence and limits

All 56 Python files passed syntax parsing. The updated source contains 124 test functions. These checks do not establish that the tests pass. The required application and testing dependencies were unavailable, so no application tests, PostgreSQL concurrency tests or runtime attack reproductions were executed for this review. Live GitHub CI was not verified.

The conclusions below come from the inspected implementation and test definitions. The review is targeted to the five earlier findings and is not an exhaustive security assessment. Code line references apply only to this snapshot.

BGP F01 Protect factor replacement

High | REQ-002 and REQ-038 | Partially resolved

Changes observed

Login now separates password verification from the second-factor challenge. Approval checks use session-level MFA assurance, and successful factor verification or recovery changes the token version to invalidate older sessions.

Remaining gap

The enroll function immediately overwrites the existing secret and sets `mfa_enabled` to `False`. The old factor stops protecting new logins before its replacement has been verified. During that interval, a person with the password can obtain a full password-only session. Because the account now appears unenrolled, that session can also replace the pending secret. The interval persists if the legitimate user abandons setup.

Effect

An existing MFA-protected account can temporarily lose its second-factor protection during replacement. This is a source-supported control gap; it has not been reproduced at runtime in this review.

Required correction

Keep the active factor enabled and store the proposed replacement separately. Bind replacement setup to the authorised session and expire abandoned requests. Verify the new factor before atomically switching secrets, changing the token version and invalidating affected sessions. Cancelling setup must leave the active factor intact.

Verification before closure

1. Start replacement on an enrolled account, then attempt password-only login from another client. A second factor must still be required.
2. A password-only or stale session cannot overwrite another session's pending replacement.
3. Failed, expired or cancelled replacement preserves the original working factor.
4. Successful replacement activates only the verified new factor and invalidates the affected old sessions.

Source locations

backend/app/routers/mfa.py, enroll and verify; backend/app/routers/auth.py, login; backend/app/deps.py, require_mfa.

BGP F02 Use preparation history for independence

High | REQ-006 | Partially resolved

Changes observed

The deciding user now references a stored compensating review created by the reviewer's own authenticated, MFA-verified action. Decision processing checks the occurrence, manifest and current reviewer authority.

Remaining gap

The self-only approval check still builds the preparer set from current owner_user_id fields. If this set is empty or differs from the deciding actor alone, the function returns without requiring a compensating review. Evidence revision requests can change or clear ownership while recording the actual editing actor separately.

Effect

A person who prepared the required evidence can reassign or clear ownership and then avoid the independent-review requirement. Changing responsibility does not establish that a different person prepared or reviewed the evidence.

Required correction

Determine preparation from the attributed revisions included in the reviewed manifest, using an explicit rule for which contributions count. Keep current ownership as a responsibility field. Recheck reviewer independence against that preparation history when committing the decision.

Verification before closure

1. A sole preparer who clears ownership must still be prevented from approving without an independent review.
2. Reassigning evidence to a colleague who did not prepare it must not satisfy separation of duties.
3. A valid review by an authorised independent person permits the intended scoped approval.
4. Changed evidence, a review for another occurrence or revoked reviewer authority is rejected.

Source locations

backend/app/routers/decisions.py, _check_separation_of_duties, especially lines 311-313;
create_compensating_review. backend/app/routers/projects.py, create_evidence_revision.

BGP F03 Complete transactional protection

High | REQ-024 and REQ-023 | Partially resolved

Changes observed

Evidence writes and decision readiness reads now lock evidence rows. Decision authority locks the caller's project-membership row. Migration 0011 adds unique constraints for one root decision per occurrence and one successor per superseded decision.

Remaining gap

Exception validity, tenant membership and compensating reviewer authority are still read without equivalent coordinated protection. A relevant revocation can therefore race the decision. Separately, the explicit decision flush occurs before the try block that catches IntegrityError during commit. A concurrent root or superseding decision can hit the new unique constraint during that earlier flush and bypass the intended conflict handler.

Effect

Approval can rely on inputs that change during the transaction, and a losing concurrent request can receive an unhandled error instead of a defined conflict response. These are source-identified risks; the precise overlapping execution was not reproduced here.

Required correction

Extend the transaction protocol to every mutable input on which approval depends, including relevant authority and exceptions. Place both flush and commit inside the intended conflict handling and restore the required tenant context after rollback before querying the winning result. Exercise the actual request paths against PostgreSQL.

Verification before closure

1. Use controlled overlapping transactions to revoke a relied-on exception or reviewer authority during approval. The result must serialize safely or reject stale authority.
2. Race two root decisions and two supersessions through the endpoints. Assert one valid result and a deterministic response for the loser, with no unhandled database error.
3. Verify retries return the intended durable result and cannot create duplicate history.
4. Run the application-path tests with the relevant lock or error handling removed and confirm they detect the defect.

Source locations

backend/app/routers/decisions.py, _compute_readiness, _check_separation_of_duties, _require_decision_authority and _record_decision; flush at line 553 and commit handler at line 578.
backend/tests/test_bgp_f03_decision_concurrency.py.

BGP F04 Preserve original historical meaning

High | REQ-030 and REQ-019 | Partially resolved

Changes observed

Exports now include complete evidence revision lists, source versions and hashes, fuller decision details and compensating reviews. Imported historical records are included in subsequent exports.

Remaining gap

The imported EvidenceRevision constructor does not restore the exported created_at timestamp. When an original author cannot be matched locally, the revision is attributed to the importing user. Subsequent revision exports use those live fields, changing the recorded author and time. Historical records with missing references may also be skipped while the import is marked committed.

Effect

A transfer can preserve record counts while changing who did the work and when it happened. A committed import can omit decision or audit history. These behaviours do not establish the required lossless transfer, even though the original uploaded archive remains available in the import job.

Required correction

Preserve original author identity and timestamps as historical provenance separately from local account mappings. Re-export that provenance without granting imported identities access. Validate required historical relationships and fail atomically when they are incomplete. Any deliberately partial import needs explicit user-visible classification and must not be claimed as lossless completion.

Verification before closure

1. Import multiple revisions into a clean instance with unmatched authors. Compare original authors, timestamps, source hashes and revision numbers after re-export.
2. Repeat the export and import cycle and compare full semantic history, including decision manifests and cross-record links.
3. A missing required historical reference produces a clear rejection with no partial project, or an explicitly authorised partial result that is not labelled lossless.
4. Historical identity records never create login credentials, active memberships or approval authority.

Source locations

backend/app/routers/exports.py, _build_archive and commit_import; revision construction at lines 552–563 and historical reference handling at lines 573–609.

Setup verification and closure record

BGP F05 Setup documentation

Priority Medium. Status Substantially addressed with verification pending. README.md now describes the implemented backend, database roles, environment variables, migration steps, persistent keys, tests and known limitations. This resolves the main stale-status issue. The clean installation and restart procedure was not executed, so the finding remains open for verification.

Before closure, follow the README from a clean checkout, apply migrations using the owner role, start the service using the application role, and run the documented suites. Record passed and skipped tests separately. Confirm that restarting with the documented persistent keys preserves access to existing development MFA secrets. Check the sequence of working-directory instructions while performing the setup.

Required evidence for closure

For each finding, record the responsible person, remediation commit, regression test names, environment, execution results, evidence location, verifier and closure date. Move an implemented fix to ready for verification before closing it. The recommended first priorities are MFA replacement and preparation attribution, followed by complete decision transaction protection and lossless history transfer.

The concurrency tests currently demonstrate raw database lock blocking and uniqueness enforcement. Those checks are useful, but they do not verify the application's complete transaction and HTTP error paths. Add application-level PostgreSQL tests for those paths before treating F03 as closed.

Source identity

Reviewed archive: pm-sdlc-tracker-bgp-master(1).zip. ZIP comment commit:

b1fb402bd50405b32d7d37084e71fdb8f79a4eff. This identifier is archive metadata and was not independently checked against GitHub.

Archive SHA-256: 8a1ed4d1d401143011ee0c72970a4038844d19fb5bd7e0a14e2a599191d8371f

Comparison source: pm-sdlc-tracker-bgp-master.zip and BGP Development Review Findings v1.0, dated 17 September 2026. The follow-up inspected changed implementation paths and related tests; no source code or tracker status was changed.

Authority and scope

This report does not change Project Directive v1.2 Approved, Blueprint v0.2 Approved or the separate KenAddme institutional frameworks. Requirement identifiers carry forward the original review mappings. It does not authorise production, certify security or record a completed independent human review. The absence of a frontend remains an acknowledged implementation limitation.